

Extending Conditionally Borrowed

Document #: P3117R1
Date: 2024-12-14
Project: Programming Language C++
Audience: SG-9
LEWG
Reply-to: Zach Laine
<whatwasthataddress@gmail.com>
Jonathan Müller
<jonathanmueller.dev@gmail.com>
Barry Revzin
<barry.revzin@gmail.com>

Contents

- 1 Changelog
 - 1.1 Changes since R0
- 2 Motivation
- 3 Why care about `borrowed_range`?
- 4 Status quo
- 5 Proposed change
 - 5.1 Create implementation-only trait
 - 5.2 `transform_view`
 - 5.2.1 ABI concerns
- 6 Implementation experience
- 7 Costs versus benefits
- 8 References

§ 1 Changelog

§ 1.1 Changes since R0

- Remove changes for all views except `transform_view`.
- Completely change the strategy of the changes for maximum ABI compatibility.

§ 2 Motivation

In [P2017R1], we made some range adaptors conditionally borrowed. But we didn't touch adaptors that had callables - like `views::transform`. It turns out to be very useful to have a borrowable version of `views::transform`. Indeed, P2728R6 even adds a dedicated new range adaptor (`views::project`) which is simply a version of `views::transform` that can be borrowed (because its callable must be a constant).

But rather than add a dedicated view for this specific case, which requires a new name but really only helps `views::transform`, we can generalize `views::transform` to address the use-case in a way that would also help all the other range adaptors that take callables. At the very least, in `views::transform(r, f)` if `r` is borrowed and `f` is empty, an implementation can simply put `f` in the `transform_view<R, F>::iterator` directly (rather than a `transform_view<R, F> *`) which would allow it to be borrowed. The same could be said for other range adaptors that take callables as well, which seems like a more useful approach as well as not requiring new names for every adaptor.

The main question then is what the criteria should be for when `transform_view<R, F>` should be a borrowed range (when `R` is):

- `is_empty_v<F>` (`range-v3` already does this - not for conditionally borrowed, but just to decide whether to store the callable by value in the iterator)
- `sizeof(F) <= sizeof(void *)` and `is_trivially_copyable_v<F>` (this means that when transforming with a function pointer, the function pointer itself can live in the iterator - which takes the same amount of space as the parent pointer, except with one less indirection)
- something else?

This question is a little simpler for `views::transform` (which only needs to potentially store `f` in the adapted iterator) than it is for `views::filter` (which would need not only the predicate but also the underlying sentinel, so this may not be worthwhile). This would need to be carefully considered.

§ 3 Why care about `borrowed_range`?

`borrowed_range` is important because it allows the standard range adaptors and views to know that a particular view is a proper view – that is, a reference type, not an expensive-to-copy type like a `std::vector<int>`. Certain views refuse to work with non-`borrowed_range` ranges, because the result might dangle. For instance, you cannot construct a subrange from a non-`borrowed_range`. In other cases, an rvalue range might be wrapped in an `owning_view` to turn it into a view, if it was not already a `borrowed_range`. So `std::vector<int>({1,2,3,4,5,6,7,8,9}) | take(3)` would take the rvalue vector, wrap it in an `owning_view`, and the result is a non-`borrowed_range` that is move-only.

Here is the part of 25.4.2 [range.range] that describes borrowed_range:

```
template<class T>
concept borrowed_range =
    range<T> && (is_lvalue_reference_v<T> ||
    enable_borrowed_range<remove_cvref_t<T>>);
```

- 4 Let U be $\text{remove_reference_t}<T>$ if T is an rvalue reference type, and T otherwise. Given a variable u of type U , T models `borrowed_range` only if the validity of iterators obtained from u is not tied to the lifetime of that variable.
- 5 [Note 2: Since the validity of iterators is not tied to the lifetime of a variable whose type models `borrowed_range`, a function with a parameter of such a type can return iterators obtained from it without danger of dangling. ? end note]

To increase the interoperability of the templates in `std::ranges` and to avoid unnecessary wrapping, we should make as many views `borrowed_ranges` as is reasonable. It is probably unreasonable to do so if there is a significant cost involved. To make a view a `borrowed_range` means copying its state out of the view itself, and into the iterator instead.

Note that copying state to the iterator is not sufficient to guarantee borrowed-ness, though is it necessary. Most views take an existing view V and do something with it, making a new view over the existing one. For instance, `transform_view<V, F>` creates a view by transforming the elements of V using the invocable F .

A particular view specialization is only borrowed if it satisfies the `borrowed_range` concept, *and* V does as well. This is important, because if a view is not borrowed, no view that uses it will be either. In `rng | V1 | V2 | ... | Vn`, if *any* V_i is not borrowed, the entire resulting view is not.

§ 4 Status quo

Currently, there are several views that are never `borrowed_ranges` that potentially could be. They are: `transform_view`, `zip_transform_view`, `adjacent_transform_view`, `filter_view`, `take_while_view`, `chunk_by_view`, `join_view`, `join_with_view`, `split_view`, `lazy_split_view`, and `cartesian_product_view`.

A previous version of this paper attempted to address the borrowability of several of those views. The consensus in SG-9 was that, since this would be a n ABI break for all of these views, that approach was too ambitious. This version of the paper radically scales that effort back; the plan now is to make a minimal change to `transform_view`. If that change can gain consensus in SG-9, we can proceed with more views, if possible.

§ 5 Proposed change

All these changes are in `std::ranges`.

§ 5.1 Create implementation-only trait

```
template<class F>
constexpr bool tidy_func =
    is_empty_v<F> && is_trivially_default_constructible_v<F> &&
    is_trivially_destructible_v<F>;
```

`tidy_func` evaluates to true when an invocable is a candidate for being re-created on the fly in a view's iterators.

§ 5.2 transform_view

Change `transform_view::iterator` to create a new object of the type of `transform_view`'s `F` template parameter. That is, the iterator will not reach back through the `parent_data` member to get the `fun` member out of the view. Also, add this specialization:

```
template<class T, class F>
constexpr bool enable_borrowed_range<transform_view<T>> =
    enable_borrowed_range<T> && tidy_func<F>;
```

This simple change means that, for any `transform_view` `tv` for which `enable_borrowed_range` is true, it is safe to use `tv.begin()/tv.end()` after the lifetime of `tv`. Even though `tv.begin()` contains a dangling pointer to `tv`, it will never dereference that pointer.

§ 5.2.1 ABI concerns

This still poses an ABI problem, though it is much less likely to affect real code than the change in the R0 revision of this paper. The layout and even the values of data members is unchanged in `transform_view`, `transform_view::iterator`, and `transform_view::sentinel`. So, passing `transform_views` and even their iterator/sentinel pairs across an ABI boundary should be safe in almost all cases – whichever side of the ABI boundary is using the objects should use them in the right way. There is no ODR issue with this change.

However, it is possible to construct a memory-unsafe usage across an ABI boundary. New code, seeing that `is_borrowed_range<transform_view<...>>` is true, might get rid of the view, because it can. Then old code might try to use the dangling `parent_data` pointer:

```
// old_tu.cpp
// Code built with C++20 semantics. Separately compiled, but with the
// types
// and function declarations available in a header.

auto transform_foos(std::vector<int> const & vec)
    -> decltype(vec | std::views::transform(some_f));
```

```

using transformed_foos_t =
    decltype(transform_foos(std::declval<std::vector<int> const &>()));

void    print_transformed_range(transformed_foos_t::iterator    f,
    transformed_foos_t::sentinel l) {
    for (auto it = f; it != l; ++it) {
        std::cout << *it << "\n";
    }
}

// new_tu.cpp
// Code built with post-P3117 semantics.

void new_using_func() {
    auto r = std::ranges::subrange(transform_foos(get_vec())); // leaves
        parent_ to dangle
    // Uses F{} to get the predicate in the iterator. OK.
    for (auto i: r) {
        std::cout << i << "\n";
    }

    // Runs in old_tu.cpp. Tries to use parent_>pred_, but parent_ is
    // dangling. UB.
    print_transformed_range(r.begin(), r.end());
}

```

There are a number of unlikely conditions that must exist for the user to write this unsafe code.

1. It is difficult to name the types of std views and their iterators. As you can see above, we had to make a separate function that creates the transform_view to make it easier to get a typename for it.
2. The user must pass an iterator/sentinel pair to the old TU (as above), or possibly put the iterator/sentinel pair into a subrange. Passing the entire transform_view to a function means that it will outlive its iterators. Passing a subrange is possible, but unlikely – if you wanted to pass the iterator/sentinel pair as a range, you would probably just pass the transform_view.
3. The potential danger of this case is limited to the `is_borrowed_range<Expr> == true` case, which is affected by the borrowability of all ranges and views in the view expression `Expr`. In other words, the proposed changes to transform_view will make it borrowable, but the danger above is only present if transform_view is actually used in a fully-borrowed view expression.

This kind of failure is the bad kind – it is memory unsafe, and leaves no lexical trace of its unsafety. However, it is also exceedingly unlikely that the user will find themselves in this situation.

§ 6 Implementation experience

One of the authors implemented the changes suggested above. The implementation was done by taking the libstdc++ implementations, copying them into a new header, in a new namespace under `std::ranges`, and altering them. The implementation can be found [here](#). The header is accompanied by a test file, and a small perf test. The implementation was very straightforward.

§ 7 Costs versus benefits

TODO: No real change expected; run the perf tests to be sure.

§ 8 References

[P2017R1] Barry Revzin. 2020-02-19. Conditionally borrowed ranges.
<https://wg21.link/p2017r1>